

LAPORAN PRAKTIKUM 8
PEMROGRAMAN BERORIENTASI OBJEK
ERROR HANDLING DAN JAVA I/O



Disusun Oleh:

Nama : Mandanta Guru Singa
NIM : 124140147
Mata Kuliah : Pemrograman Berorientasi Objek
Dosen Pengampu : Rahman Indra Kesuma, S. Kom., M. Cs.
Kelas : RB

PROGRAM STUDI TEKNIK INFORMATIKA
INSTITUT TEKNOLOGI SUMATERA

2026

1. Tujuan Praktikum

- Menjelaskan pengertian error dan exception dalam Java.
- Membedakan syntax error, runtime error, dan logical error.
- Menggunakan blok try-catch untuk menangani exception.
- Menggunakan blok finally dalam program.
- Memahami penggunaan sederhana throw dan throws.
- Membaca input dari pengguna menggunakan Scanner.
- Menulis data ke file teks menggunakan Java I/O.
- Membaca data dari file teks menggunakan Java I/O.
- Membuat program sederhana yang menangani error dan menyimpan data ke file.

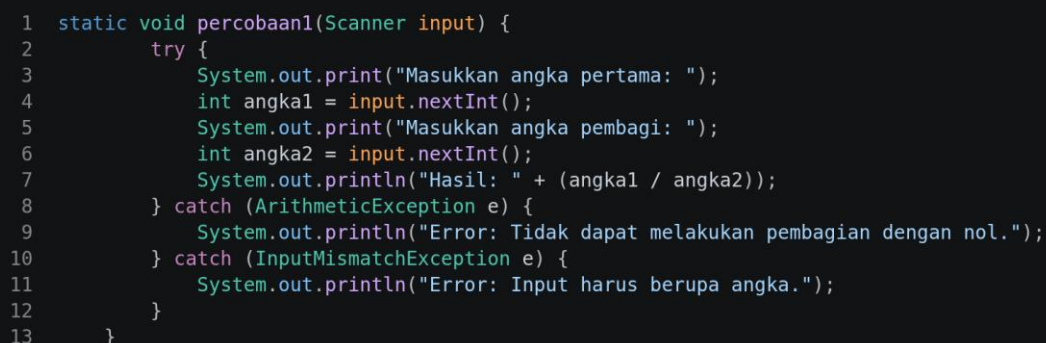
2. Alat dan Bahan

- Laptop
- VS Code
- JDK versi 25
- Modul Praktikum PBO 8

3. Langkah Percobaan

3.1. Percobaan 1: Try-Catch Dasar

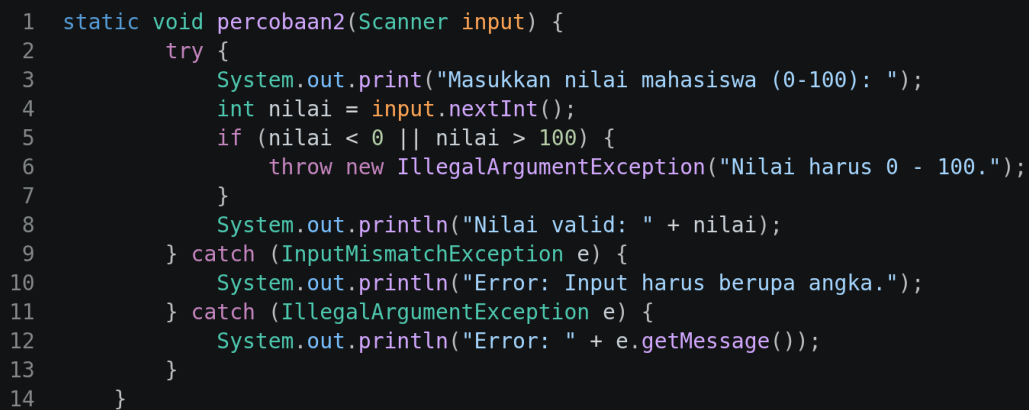
- Menerapkan blok try-catch untuk menangani ArithmeticException pada operasi pembagian dua angka agar program tidak berhenti tiba-tiba saat terjadi pembagian dengan nol.



```
1 static void percobaan1(Scanner input) {
2     try {
3         System.out.print("Masukkan angka pertama: ");
4         int angka1 = input.nextInt();
5         System.out.print("Masukkan angka pembagi: ");
6         int angka2 = input.nextInt();
7         System.out.println("Hasil: " + (angka1 / angka2));
8     } catch (ArithmeticException e) {
9         System.out.println("Error: Tidak dapat melakukan pembagian dengan nol.");
10    } catch (InputMismatchException e) {
11        System.out.println("Error: Input harus berupa angka.");
12    }
13 }
```

3.2. Percobaan 2: Validasi Input Nilai

- Menambahkan validasi input nilai menggunakan keyword throw untuk melempar IllegalArgumentException pada rentang nilai yang tidak valid, serta menangani input bukan angka menggunakan InputMismatchException.



```
1 static void percobaan2(Scanner input) {
2     try {
3         System.out.print("Masukkan nilai mahasiswa (0-100): ");
4         int nilai = input.nextInt();
5         if (nilai < 0 || nilai > 100) {
6             throw new IllegalArgumentException("Nilai harus 0 - 100.");
7         }
8         System.out.println("Nilai valid: " + nilai);
9     } catch (InputMismatchException e) {
10        System.out.println("Error: Input harus berupa angka.");
11    } catch (IllegalArgumentException e) {
12        System.out.println("Error: " + e.getMessage());
13    }
14 }
```

3.3. Percobaan 3: Menulis dan Membaca File

- Menggunakan kelas FileWriter, PrintWriter, dan Scanner untuk menulis input data pengguna ke dalam file mahasiswa.txt lalu membacanya kembali, lengkap dengan penanganan IOException dan FileNotFoundException.

```

1  static void percobaan3(Scanner input) {
2      try {
3          System.out.print("Masukkan Nama: ");
4          String nama = input.nextLine();
5          System.out.print("Masukkan NIM: ");
6          String nim = input.nextLine();
7
8          FileWriter fw = new FileWriter("mahasiswa.txt");
9          PrintWriter pw = new PrintWriter(fw);
10         pw.println("Nama: " + nama);
11         pw.println("NIM: " + nim);
12         pw.close();
13         System.out.println("Data berhasil disimpan ke mahasiswa.txt");
14
15         Scanner bacaFile = new Scanner(new File("mahasiswa.txt"));
16         System.out.println("\nIsi file:");
17         while (bacaFile.hasNextLine()) {
18             System.out.println(bacaFile.nextLine());
19         }
20         bacaFile.close();
21     } catch (IOException e) {
22         System.out.println("Terjadi kesalahan I/O saat memproses file.");
23     }
24 }

```

3.4. Percobaan 4: Studi Kasus Mini (Sistem Data Mahasiswa)

- Menggabungkan operasi validasi data dan manipulasi file ke dalam satu fungsi terpadu menggunakan blok try-catch-finally untuk memastikan sumber daya tertutup dengan benar.

```

1  static void percobaan4(Scanner input) {
2      try {
3          System.out.print("Masukkan prodi: ");
4          String prodi = input.nextLine();
5          System.out.print("Masukkan semester: ");
6          int semester = input.nextInt();
7
8          if (prodi.isEmpty()) throw new IllegalArgumentException("Prodi tidak boleh kosong.");
9          if (semester < 1) throw new IllegalArgumentException("Semester minimal 1.");
10
11          FileWriter fw = new FileWriter("data_mahasiswa.txt", true);
12          PrintWriter pw = new PrintWriter(fw);
13          pw.println("Prodi: " + prodi + ", Semester: " + semester);
14          pw.close();
15          System.out.println("Data ditambahkan ke data_mahasiswa.txt");
16
17      } catch (IllegalArgumentException e) {
18          System.out.println("Kesalahan validasi: " + e.getMessage());
19      } catch (IOException e) {
20          System.out.println("Gagal menyimpan data ke file.");
21      } catch (InputMismatchException e) {
22          System.out.println("Kesalahan input: Semester harus berupa angka.");
23      } finally {
24          System.out.println("Sesi pembersihan resource (finally) dieksekusi.");
25      }
26  }

```

- Full Code

```

1 import java.io.File;
2 import java.io.FileWriter;
3 import java.io.IOException;
4 import java.io.PrintWriter;
5 import java.util.InputMismatchException;
6 import java.util.Scanner;
7
8 public class Main {
9
10     public static void main(String[] args) {
11         Scanner scanner = new Scanner(System.in);
12
13         System.out.println("=== Eksekusi Percobaan 1 ===");
14         percobaan1(scanner);
15         scanner.nextLine(); // Membersihkan buffer enter sebelum lanjut
16
17         System.out.println("=== Eksekusi Percobaan 2 ===");
18         percobaan2(scanner);
19         scanner.nextLine(); // Membersihkan buffer enter sebelum lanjut
20
21         System.out.println("=== Eksekusi Percobaan 3 ===");
22         percobaan3(scanner);
23
24         System.out.println("=== Eksekusi Percobaan 4 ===");
25         percobaan4(scanner);
26
27         scanner.close();
28         System.out.println("Semua percobaan selesai.");
29     }
30
31     static void percobaan1(Scanner input) {
32         try {
33             System.out.print("Masukkan angka pertama: ");
34             int angka1 = input.nextInt();
35             System.out.print("Masukkan angka pembagi: ");
36             int angka2 = input.nextInt();
37             System.out.println("Hasil: " + (angka1 / angka2));
38         } catch (ArithmeticException e) {
39             System.out.println("Error: Tidak dapat melakukan pembagian dengan nol.");
40         } catch (InputMismatchException e) {
41             System.out.println("Error: Input harus berupa angka.");
42         }
43     }
44
45     static void percobaan2(Scanner input) {
46         try {
47             System.out.print("Masukkan nilai mahasiswa (0-100): ");
48             int nilai = input.nextInt();
49             if (nilai < 0 || nilai > 100) {
50                 throw new IllegalArgumentException("Nilai harus 0 - 100.");
51             }
52             System.out.println("Nilai valid: " + nilai);
53         } catch (InputMismatchException e) {
54             System.out.println("Error: Input harus berupa angka.");
55         } catch (IllegalArgumentException e) {
56             System.out.println("Error: " + e.getMessage());
57         }
58     }
59
60     static void percobaan3(Scanner input) {
61         try {
62             System.out.print("Masukkan Nama: ");
63             String nama = input.nextLine();
64             System.out.print("Masukkan NIM: ");
65             String nim = input.nextLine();
66
67             FileWriter fw = new FileWriter("mahasiswa.txt");
68             PrintWriter pw = new PrintWriter(fw);
69             pw.println("Nama: " + nama);
70             pw.println("NIM: " + nim);
71             pw.close();
72             System.out.println("Data berhasil disimpan ke mahasiswa.txt");
73
74             Scanner bacaFile = new Scanner(new File("mahasiswa.txt"));
75             System.out.println("Isi file:");
76             while (bacaFile.hasNextLine()) {
77                 System.out.println(bacaFile.nextLine());
78             }
79             bacaFile.close();
80         } catch (IOException e) {
81             System.out.println("Terjadi kesalahan I/O saat memproses file.");
82         }
83     }
84
85     static void percobaan4(Scanner input) {
86         try {
87             System.out.print("Masukkan prodi: ");
88             String prodi = input.nextLine();
89             System.out.print("Masukkan semester: ");
90             int semester = input.nextInt();
91
92             if (prodi.isEmpty()) throw new IllegalArgumentException("Prodi tidak boleh kosong.");
93             if (semester < 1) throw new IllegalArgumentException("Semester minimal 1.");
94
95             FileWriter fw = new FileWriter("data_mahasiswa.txt", true);
96             PrintWriter pw = new PrintWriter(fw);
97             pw.println("Prodi: " + prodi + ", Semester: " + semester);
98             pw.close();
99             System.out.println("Data ditambahkan ke data_mahasiswa.txt");
100         } catch (IllegalArgumentException e) {
101             System.out.println("Kesalahan validasi: " + e.getMessage());
102         } catch (IOException e) {
103             System.out.println("Gagal menyimpan data ke file.");
104         } catch (InputMismatchException e) {
105             System.out.println("Kesalahan input: Semester harus berupa angka.");
106         } finally {
107             System.out.println("Sesi pembersihan resource (finally) dieksekusi.");
108         }
109     }
110 }
111

```

4. Source Code

<https://github.com/DanamiStartrail/Tugas-Laporan-PBO/tree/Pertemuan-8>

5. Hasil Output Program

```
=== Eksekusi Percobaan 1 ===
Masukkan angka pertama: input teks
Error: Input harus berupa angka.

=== Eksekusi Percobaan 2 ===
Masukkan nilai mahasiswa (0-100): -1
Error: Nilai harus 0 - 100.

=== Eksekusi Percobaan 3 ===
Masukkan Nama:
Masukkan NIM:
Data berhasil disimpan ke mahasiswa.txt

Isi file:
Nama:
NIM:

=== Eksekusi Percobaan 4 ===
Masukkan prodi: Informatika
Masukkan semester: 17
Data ditambahkan ke data_mahasiswa.txt
Sesi pembersihan resource (finally) dieksekusi.

Semua percobaan selesai.
```

6. Analisis Program

Program Java ini menerapkan konsep Error Handling dan Java I/O secara terpadu untuk memodelkan sistem penerimaan dan penyimpanan data yang aman. Struktur kode terdiri dari kelas utama yang mengeksekusi beberapa tahap percobaan, dengan memanfaatkan kelas bawaan Java seperti Scanner, File, FileWriter, dan PrintWriter.

Program ini mengimplementasikan validasi dan persistensi data secara berjenjang. Alurnya adalah: Membaca input pengguna → Memvalidasi logika data → Menyimpan data ke *file* eksternal → Membaca kembali data dari *file*. Hal ini membuat program tidak hanya sekadar memproses data di memori, tetapi memastikan data tersebut utuh dan tersimpan secara fisik di *storage*.

Penggunaan blok try-catch-finally dan *keyword* throw terlihat pada setiap metode. Misalnya, pada percobaan utama, try membungkus operasi input dan tulis/baca *file* yang rawan mengalami kegagalan. Jika terdapat input kosong atau nilai minus,

program secara eksplisit menggunakan `throw new IllegalArgumentException` untuk menciptakan penolakan manual. Sementara itu, `InputMismatchException` atau `IOException` akan ditangkap oleh blok `catch` agar program tidak *crash*. Di akhir proses, `finally` menjamin eksekusi perintah pembersihan atau pesan terminasi tetap berjalan.

Pada bagian main, eksekusi dilakukan secara sekuensial melalui satu objek `Scanner` yang dioperkan ke berbagai method. Program ini membuktikan kemampuannya untuk bertahan dari interaksi pengguna yang keliru (memasukkan teks pada input angka atau membagi dengan nol) sekaligus berhasil memodifikasi serta membaca struktur sistem berkas secara dinamis.

7. Pertanyaan Diskusi

7.1. Apa yang dimaksud dengan exception dalam Java?

Exception adalah kesalahan yang terjadi saat program sedang berjalan yang menyebabkan program tidak berjalan sesuai harapan.

7.2. Apa perbedaan syntax error, runtime error, dan logical error?

- **Syntax error:** Kesalahan yang terjadi karena penulisan kode tidak sesuai aturan bahasa Java dan terdeteksi sebelum dijalankan.
- **Runtime error:** Kesalahan yang muncul saat program sedang dijalankan meskipun berhasil dikompilasi.
- **Logical error:** Kesalahan logika di mana program berjalan tanpa error, tetapi hasil yang diberikan tidak sesuai dengan yang diharapkan.

7.3. Apa fungsi blok try dalam error handling?

Tempat untuk meletakkan kode-kode yang berpotensi menimbulkan *error* atau *exception*.

7.4. Apa fungsi blok catch dalam error handling?

Tempat untuk meletakkan kode yang bertugas menangani *error* tersebut agar program tidak langsung berhenti.

7.5. Kapan blok finally dijalankan?

Blok `finally` akan selalu dijalankan pada akhir proses penanganan, baik jika terjadi *exception* maupun tidak.

7.6. Apa perbedaan throw dan throws?

`throw` digunakan untuk melempar exception secara manual di dalam blok kode (biasanya untuk aturan validasi), sedangkan `throws` digunakan pada deklarasi method untuk memberi tahu bahwa method tersebut berpotensi menghasilkan exception.

7.7. Mengapa membaca file perlu menggunakan error handling? Karena operasi membaca file sangat rentan terhadap kondisi eksternal, contohnya file yang ingin dibaca bisa saja tidak ditemukan. Error handling diperlukan agar kesalahan seperti ini tidak menyebabkan program *crash*.

7.8. Apa manfaat menyimpan data program ke file? Agar data input maupun hasil komputasi tidak hilang setelah program ditutup, sehingga data tersebut tersimpan secara permanen (persisten).

8. Kesimpulan

Berdasarkan hasil analisis program dan diskusi mengenai konsep Error Handling dan Java I/O pada Modul 8, dapat disimpulkan bahwa:

- **Stabilitas Program melalui Try-Catch:** Error handling memungkinkan pembuatan aplikasi yang tahan terhadap gangguan tak terduga (*robust*). Kegagalan seperti kesalahan format input atau ketidaktersediaan *file* dapat dicegat dan dialihkan menjadi pesan informatif, sehingga alur eksekusi tidak berhenti secara paksa.
- **Validasi Logika Bisnis dengan Keyword throw:** Penerapan pelemparan *exception* secara manual menunjukkan bahwa sistem dapat memaksakan aturan khusus yang tidak diakomodasi oleh tipe data dasar. Pelemparan error ini menjamin integritas data—seperti menolak *string* kosong atau angka negatif—sebelum disimpan ke dalam sistem.
- **Keamanan Eksekusi Akhir dengan finally:** Penggunaan blok *finally* memastikan adanya area kode yang kebal terhadap interupsi. Ini sangat penting untuk menjaga kebersihan *resource*, memastikan bahwa proses penutupan jalur I/O akan selalu tereksekusi tanpa memedulikan apakah blok *try* berhasil atau gagal.
- **Persistensi Data Melalui Java I/O:** Kombinasi *FileWriter* dan pembaca *File* memperluas utilitas program untuk menyimpan informasi secara permanen di luar siklus hidup aplikasi. Objek dan data teks tidak akan menguap setelah program ditutup, memberikan sifat persisten yang krusial bagi sebuah sistem basis data sederhana.
- **Fondasi Aplikasi Dunia Nyata:** Pemahaman terintegrasi mengenai manajemen kesalahan (Error Handling) dan komunikasi sistem eksternal (I/O) adalah prasyarat fundamental. Tanpa fondasi ini, pembuatan perangkat lunak yang aman untuk dioperasikan oleh pengguna awam dan terhubung dengan penyimpanan data tidak akan bisa dilakukan.

Secara keseluruhan, program ini telah mendemonstrasikan bagaimana Error Handling dan Java I/O digunakan untuk menciptakan struktur kode yang andal, aman dari gangguan input, dan mampu memelihara rekam jejak data secara permanen.

